Recursion & Backtracking

Subsets, permutations, N-Queens, and the art of pruning

Learning goals

- Write recursive functions with a clear **base case** and **progress**
- See every brute force as a walk over a **recursion tree**
- Generate all **subsets** (2^n) and **permutations** ($n!$) correctly
- Apply the **choose → explore → un-choose** backtracking template
- Solve **N-Queens** and constraint puzzles with **pruning**
- Estimate running time by counting nodes in the search tree

Motivation: CSES 1624 — Chessboard and Queens

Place 8 queens on an 8×8 board so that no two attack each other.
Some squares are **reserved** (`*`) and cannot hold a queen.
Count the valid placements.

```
........
........
..*.....
........
........
........
.....**.
...*....
........
```

answer: 65

Naive: choose 8 of 64 squares → C(64,8) ≈ $4.4 \cdot 10^9$. Too slow.
Backtracking: only **2,057** recursive calls on an empty board.

Recursion refresher

A recursive function needs:

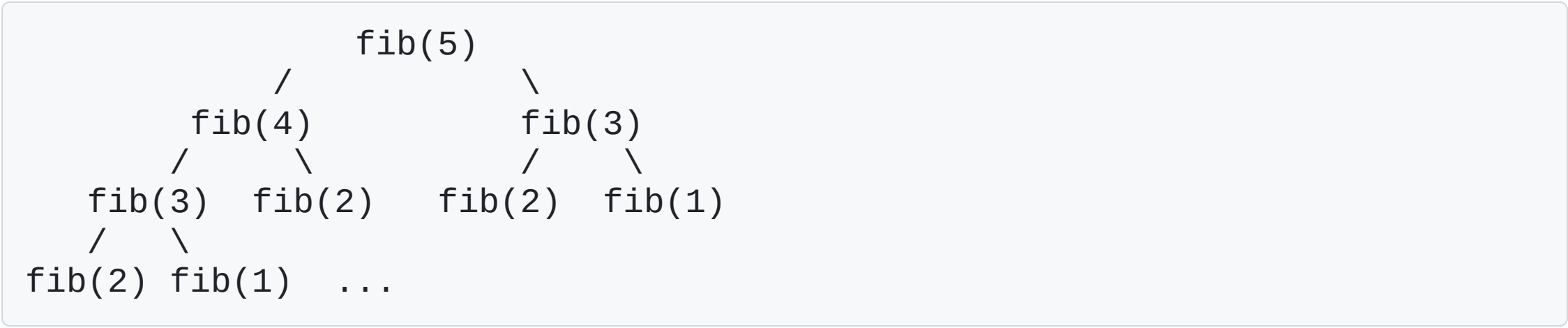
1. **Base case** — answers the smallest instance directly
2. **Recursive case** — reduces to *strictly smaller* instances

```
long long fact(int n) {
    if (n == 0) return 1;          // base case
    return n * fact(n - 1);        // smaller instance
}
```

Every call gets its own stack frame (locals, return address).
Default stack is ~8 MB on most judges (1 MB on some Windows judges):
depth 10^5–10^6 is usually fine, 10^7 is not.

Thinking in recursion trees

`fib(5)` with plain recursion:



- **Time** ≈ number of nodes × work per node
- **Memory** ≈ maximum depth (the stack)
- Repeated subtrees → memoization (Week 14)
- Here we *want* to visit every leaf: each leaf is a candidate answer

The decision-tree view of brute force

Most complete searches make a **sequence of decisions**:

Problem	Decision at level *i*	Leaves
Subsets of n items	take item *i* or not	2^n
Permutations	which unused item goes in slot *i*	n!
N-Queens	column of the queen in row *i*	$\le n^n$
Strings over Σ of length L	which letter at position *i*	$\lvert\Sigma\rvert^L$

Recursion depth = number of decisions. State = decisions made so far.

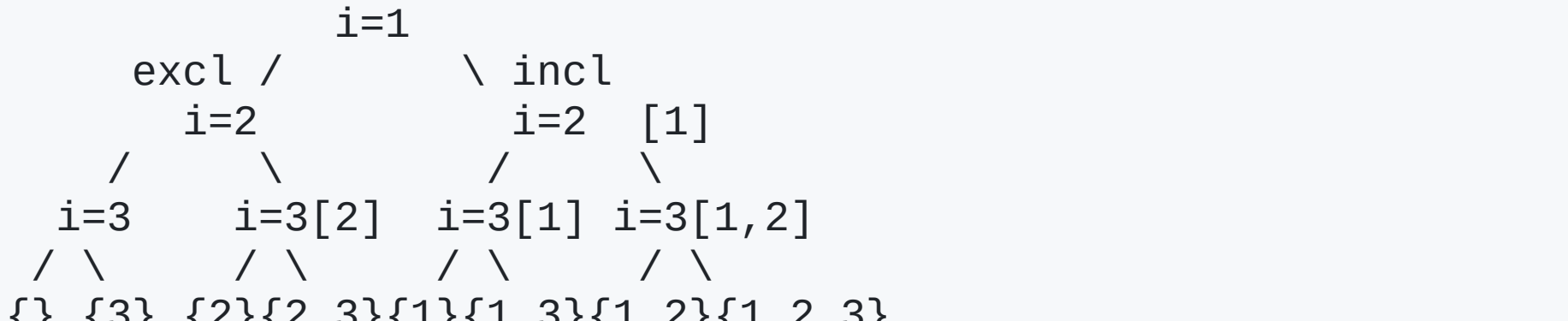
Generating subsets (include / exclude)

```
int n;
vector<int> cur;

void subsets(int i) {
    if (i == n + 1) {                // all n decisions made
        process(cur);
        return;
    }
    subsets(i + 1);                  // exclude i
    cur.push_back(i);                // choose i
    subsets(i + 1);                  // explore
    cur.pop_back();                  // un-choose
}
// call subsets(1)
```

2^n leaves, $2^{n+1} - 1$ nodes → **$O(2^n \cdot n)$** if processing is O(n).

Subsets of {1,2,3}: the tree



Output order: `{} {3} {2} {2 3} {1} {1 3} {1 2} {1 2 3}`

(Week 9 shows the iterative bitmask alternative.)

Generating permutations

```
vector<int> cur;
vector<bool> used;            // size n+1

void perms() {
    if ((int)cur.size() == n) { process(cur); return; }
    for (int v = 1; v <= n; v++) {
        if (used[v]) continue;
        used[v] = true;  cur.push_back(v);   // choose
        perms();                             // explore
        used[v] = false; cur.pop_back();     // un-choose
    }
}
```

n! leaves; total nodes ≤ e · n! → **O(n · n!)**. Feasible for n ≤ 10 (3.6M).

Or let the STL do it

```
vector<int> a = {1, 2, 3};           // must start SORTED
do {
    process(a);
} while (next_permutation(a.begin(), a.end()));
```

- Visits permutations in lexicographic order
- Handles duplicates: `{1,1,2}` → only 3 distinct permutations
- Amortized O(1) per step
- Still write the recursive version when you need **pruning** (`next_permutation` can't skip a bad prefix)

The backtracking template

```
void solve(state):
    if state is complete:
        record answer; return
    for each choice c valid in state:
        apply c                 // choose
        solve(new state)        // explore
        undo c                  // un-choose
```

The **undo** step restores shared state exactly.

Forgetting it is the #1 backtracking bug.

N-Queens: one queen per row

Decision at row *r*: which column gets the queen.

A square (r, c) is attacked if its **column** or a **diagonal** is used.

```
diagonal ↘ : r − c is constant        diagonal ↙ : r + c is constant

  c: 0 1 2 3                            c: 0 1 2 3
r0   0 -1-2-3                 r0         0 1 2 3
r1   1 0 -1-2                 r1         1 2 3 4
r2   2 1 0 -1                 r2         2 3 4 5
```

Track `col[c]`, `d1[r+c]`, `d2[r−c+n−1]` → O(1) safety check.

N-Queens code (CSES 1624)

```
string g[8];
bool colUsed[8], d1[15], d2[15];
long long ways = 0;

void place(int r) {
    if (r == 8) { ways++; return; }
    for (int c = 0; c < 8; c++) {
        if (g[r][c] == '*' || colUsed[c] || d1[r + c] || d2[r - c + 7])
            continue;                                   // prune
        colUsed[c] = d1[r + c] = d2[r - c + 7] = true;  // choose
        place(r + 1);                                   // explore
        colUsed[c] = d1[r + c] = d2[r - c + 7] = false; // un-choose
    }
}
```

Empty board → 92 solutions. Sample → 65.

Why pruning matters

Search	Nodes for 8×8
Any 8 of 64 squares	$4.4 \cdot 10^9$
One per row (8^8)	$1.7 \cdot 10^7$
One per row + column (8!)	40,320
Row + column + diagonals, checked early	2,057 calls

Prune as early as possible: reject a partial solution the moment it can no longer be completed. A pruned node removes its entire subtree.

Kinds of pruning

- **Feasibility**: partial state already breaks a constraint (queens attack)
- **Bound**: even the best completion can't beat the best answer found (branch & bound — e.g. current cost ≥ best → return)
- **Ordering**: sort choices so that a `break` kills the rest of the loop
- **Symmetry**: empty board, even n → put the first queen in the left half, ×2
- **Dead-end detection**: e.g. CSES 1625 Grid Paths — if the path hits a wall and can turn both left and right, the grid is split → stop

Example: combination sum with sorted pruning

All multisets of candidates summing to `target` (reuse allowed).

```
void go(int start, int remaining) {
    if (remaining == 0) { print(cur); return; }
    for (int i = start; i < (int)a.size(); i++) {
        if (a[i] > remaining) break;   // a is sorted: all later fail too
        cur.push_back(a[i]);
        go(i, remaining - a[i]);       // i, not i+1 → reuse allowed
        cur.pop_back();
    }
}
```

Starting from `start` (not 0) generates each multiset **once**,
in non-decreasing order — no duplicate `{2,3}` / `{3,2}`.

Avoiding duplicate outputs

Input has repeated values, e.g. distinct subsets of `{1, 2, 2}`:

```
sort(a.begin(), a.end());
void go(int start) {
    process(cur);
    for (int i = start; i < n; i++) {
        if (i > start && a[i] == a[i - 1]) continue;  // skip twin branch
        cur.push_back(a[i]); go(i + 1); cur.pop_back();
    }
}
```

Rule: at one level of the tree, never branch on the **same value** twice.

Counting nodes = estimating time

n	2^n	n!
10	1,024	3,628,800
15	32,768	$1.3 \cdot 10^{12}$
20	1,048,576	$2.4 \cdot 10^{18}$
25	$3.4 \cdot 10^{7}$	—

Rule of thumb (≈10^8 simple ops/s):

n ≤ 20–25 → 2^n OK; n ≤ 10–11 → n! OK.

Pruning helps in practice, but bound the **worst case** before coding.

Recursion pitfalls

- **Global vs. local state**: globals are shared across calls — must undo
- **Passing `vector` by value** at every call: O(n) copy per node
- **Missing base case** → infinite recursion → stack overflow (RE/MLE)
- **Deep recursion** (10^6+ levels) can overflow the stack — convert to a loop
- Returning early from inside the loop **without undoing** the choice
- Off-by-one in base case (`i == n` vs `i == n + 1`)

Tower of Hanoi (CSES 2165)

Move n disks from peg 1 to peg 3:

```
void hanoi(int n, int from, int to, int via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);   // clear the way
    moves.push_back({from, to});   // move largest disk
    hanoi(n - 1, via, to, from);   // stack the rest on top
}
```

$T(n) = 2T(n-1) + 1 \rightarrow$ **$2^n - 1$ moves**, which is optimal.

Recursion as **divide & conquer**, not search.

Meet in the middle (preview)

n = 40 items, count subsets with sum = x (CSES 1628).
$2^{40} \approx 10^{12}$ — too slow. But:

1. Enumerate all 2^{20} subset sums of the **left half**
2. Enumerate all 2^{20} subset sums of the **right half**
3. Sort one list; for each left sum *s*, count `x - s` in the right list

O(2^(n/2) · n) — the square root of brute force.

Complexity summary

Technique	Time	Extra memory
All subsets	$O(2^n \cdot n)$	$O(n)$ stack
All permutations	$O(n! \cdot n)$	$O(n)$
N-Queens (n×n)	$\ll n!$ with pruning	$O(n)$
Hanoi	$O(2^n)$	$O(n)$
Meet in the middle	O(2^(n/2) · n)	O(2^(n/2))

Practice problems

- CSES 1622 — Creating Strings (distinct permutations)
- CSES 1623 — Apple Division (subsets, n ≤ 20)
- CSES 1624 — Chessboard and Queens
- CSES 2165 — Tower of Hanoi
- CSES 1625 — Grid Path Description (heavy pruning, challenge)
- AtCoder ABC 114 C — 755
- Codeforces 550B — Preparing Olympiad
- LeetCode 39 / 46 / 51 / 78 — Combination Sum, Permutations, N-Queens, Subsets

Summary

- Brute force = **walk a decision tree**; recursion is the natural way
- Template: **choose → explore → un-choose**
- Subsets: 2^n; permutations: n!; know the limits (≈20 and ≈10)
- **Prune early**: feasibility, bounds, ordering, symmetry
- Count nodes to estimate time before you code
- Next week: **bit manipulation** — subsets as integers, no recursion needed